

FinAI

Documento de Arquitetura, UML, Fluxos e Dependências

Aplicação financeira web com React + TypeScript no frontend e Java + Spring Boot + Spring AI no backend.

Frontend	Backend	Dados	IA	Infraestrutura
React TypeScript Vite	Java Spring Boot Spring Security	PostgreSQL JPA/Hibernate Flyway	Spring AI Tool Calling LLM	Docker Docker Compose CI/CD

Objetivo

Este documento serve como blueprint inicial do projeto. Ele descreve escopo, arquitetura, atores e casos de uso, modelo de domínio, banco de dados, fluxos críticos, integração com IA, dependências, segurança, infraestrutura, APIs e organização do repositório.

1. Visão geral e escopo

O FinAI é um sistema de gestão financeira pessoal. O usuário acompanha saldo, receitas, despesas, categorias, cartões, faturas, metas e relatórios. O diferencial é um assistente de IA integrado ao domínio financeiro, capaz de consultar dados por ferramentas controladas pelo backend.

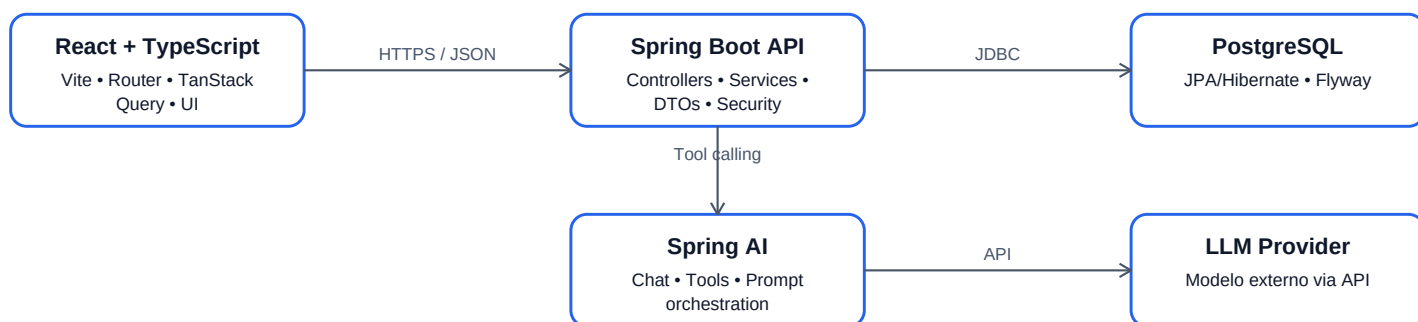
MVP funcional

- Cadastro, login e autenticação baseada em JWT.
- Conta financeira e saldo.
- Lançamentos de receitas, despesas e transferências.
- Categorias e relatórios financeiros.
- Cartão de crédito, compras e faturas.
- Metas financeiras e acompanhamento de progresso.
- Assistente FinAI com perguntas sobre os próprios dados do usuário.

Princípio de segurança

A LLM não deve acessar diretamente o banco. O backend expõe apenas ferramentas/serviços autorizados, valida o usuário e limita os dados retornados. A IA interpreta e explica; as regras financeiras permanecem no domínio da aplicação.

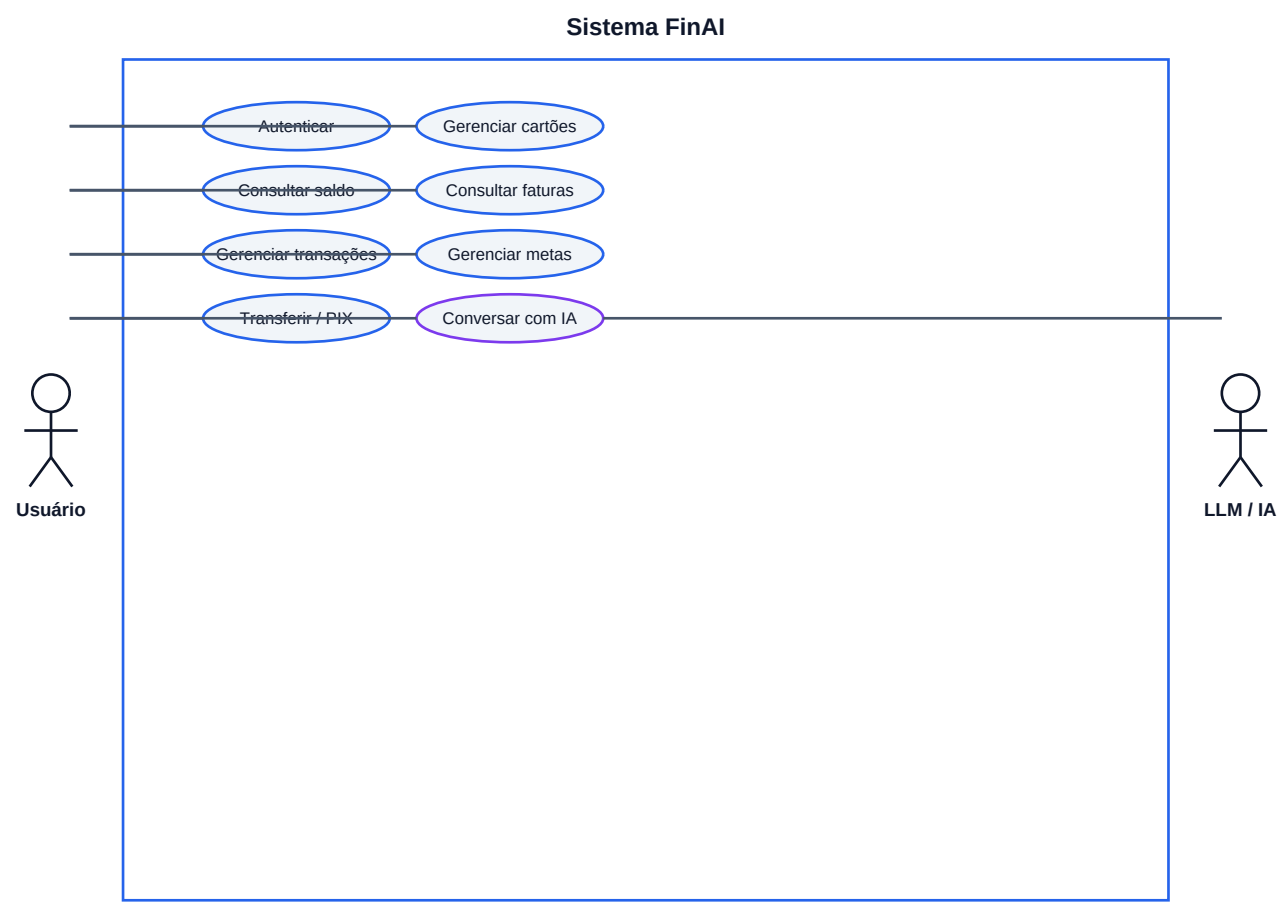
2. Arquitetura da solução



Camadas do backend

- **API**: Controllers, autenticação, validação e contratos HTTP.
- **Application**: casos de uso e orquestração das operações.
- **Domain**: entidades, enums, regras e invariantes financeiras.
- **Infrastructure**: JPA, PostgreSQL, Spring AI, integrações externas e observabilidade.

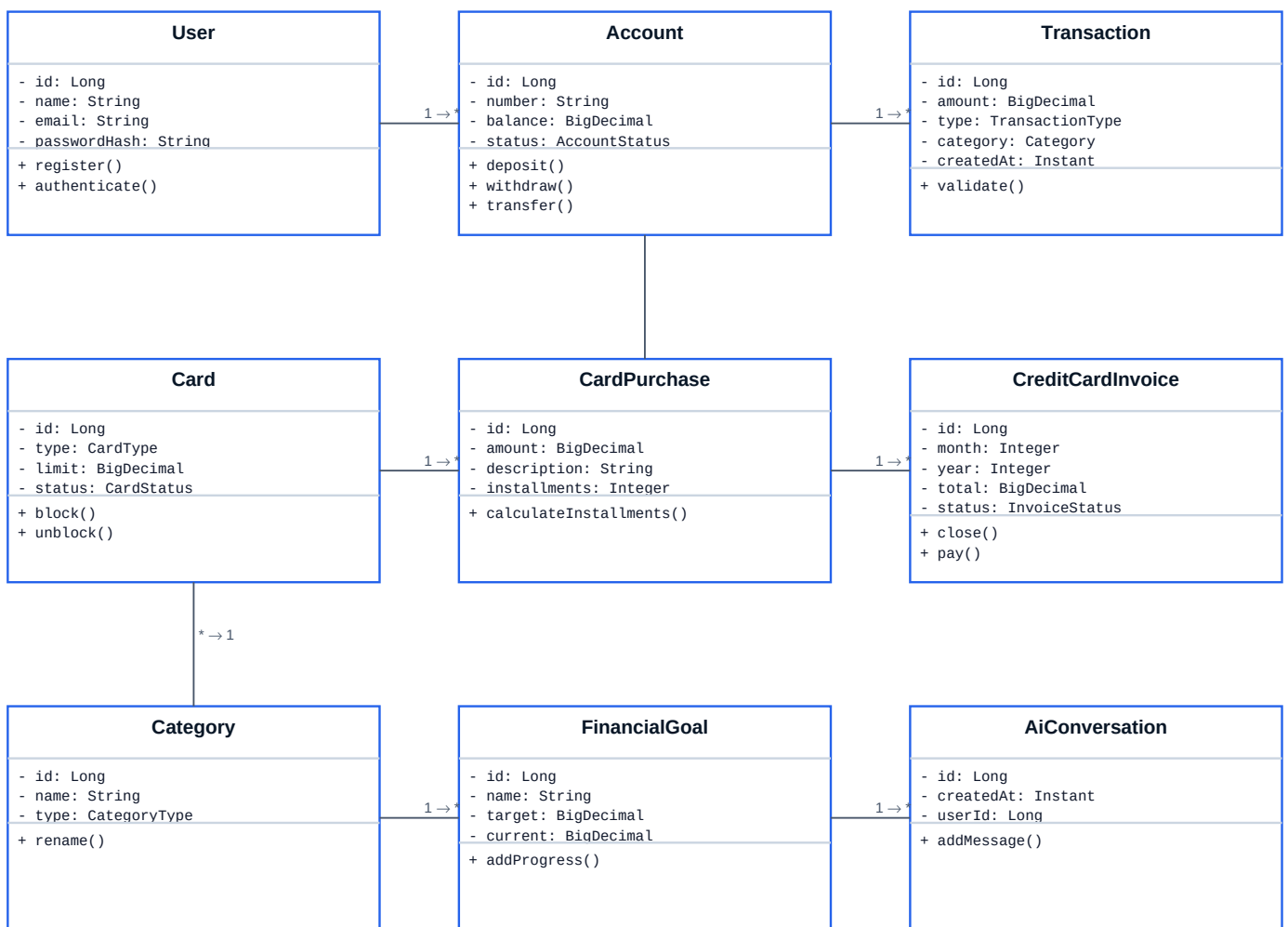
3. Diagrama de casos de uso



Casos de uso principais

ID	Caso de uso	Responsabilidade
UC01	Autenticar usuário	Login, emissão/validação de token e proteção dos recursos.
UC02	Consultar dashboard	Saldo, receitas, despesas, gráficos, metas e transações recentes.
UC03	Gerenciar transações	Criar, consultar, editar e excluir lançamentos permitidos.
UC04	Realizar transferência	Validar conta, saldo, valor e registrar débito/crédito.
UC05	Gerenciar cartões	Consultar cartão, limite, status e operações.
UC06	Gerenciar faturas	Consultar compras, parcelas, fechamento e pagamento.
UC07	Gerenciar metas	Criar metas, adicionar progresso e acompanhar evolução.
UC08	Conversar com FinAI	Perguntar, consultar ferramentas e receber resposta contextual.

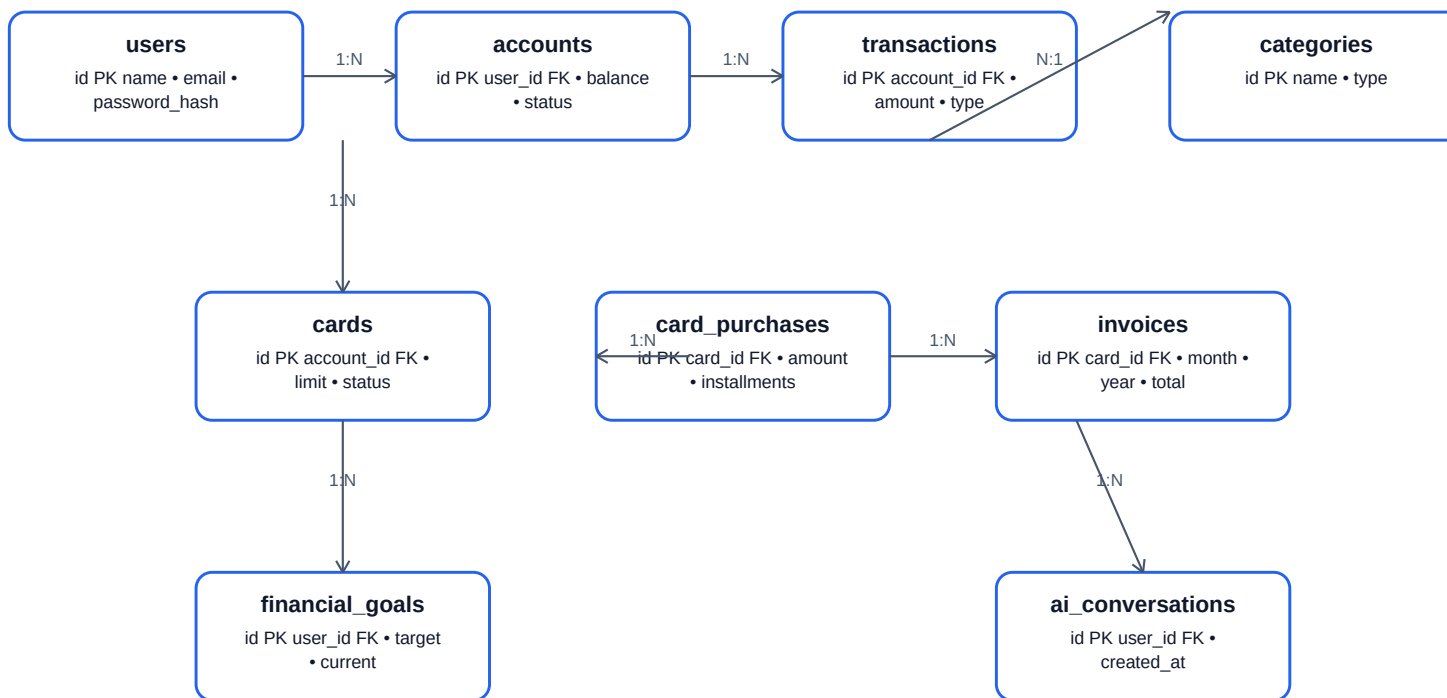
4. Diagrama de classes do domínio



Observação

O diagrama representa o núcleo do domínio. DTOs, repositories, services, controllers e classes de infraestrutura não são entidades de domínio e devem aparecer nos diagramas de arquitetura/camadas.

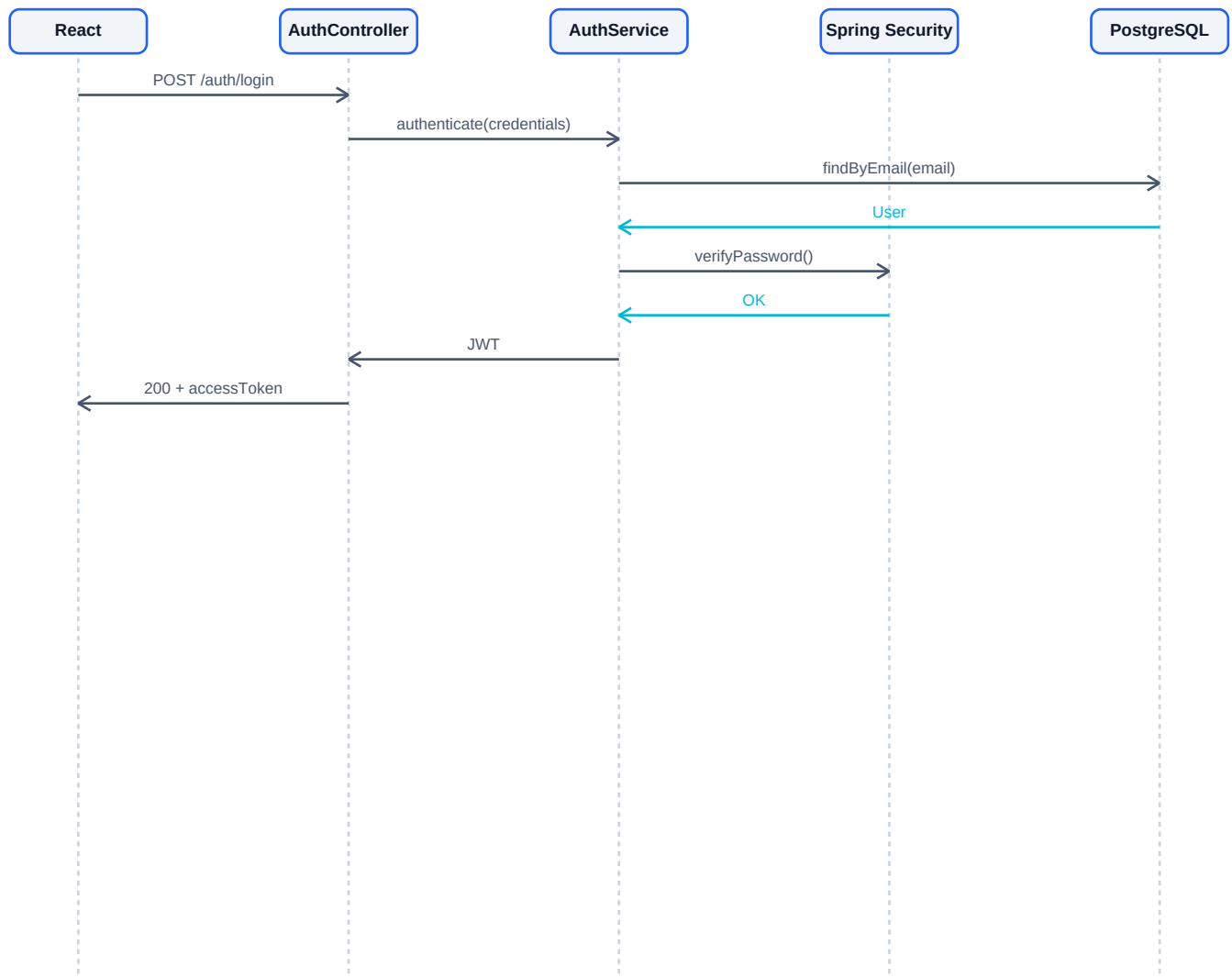
5. Modelo de dados / ER conceitual



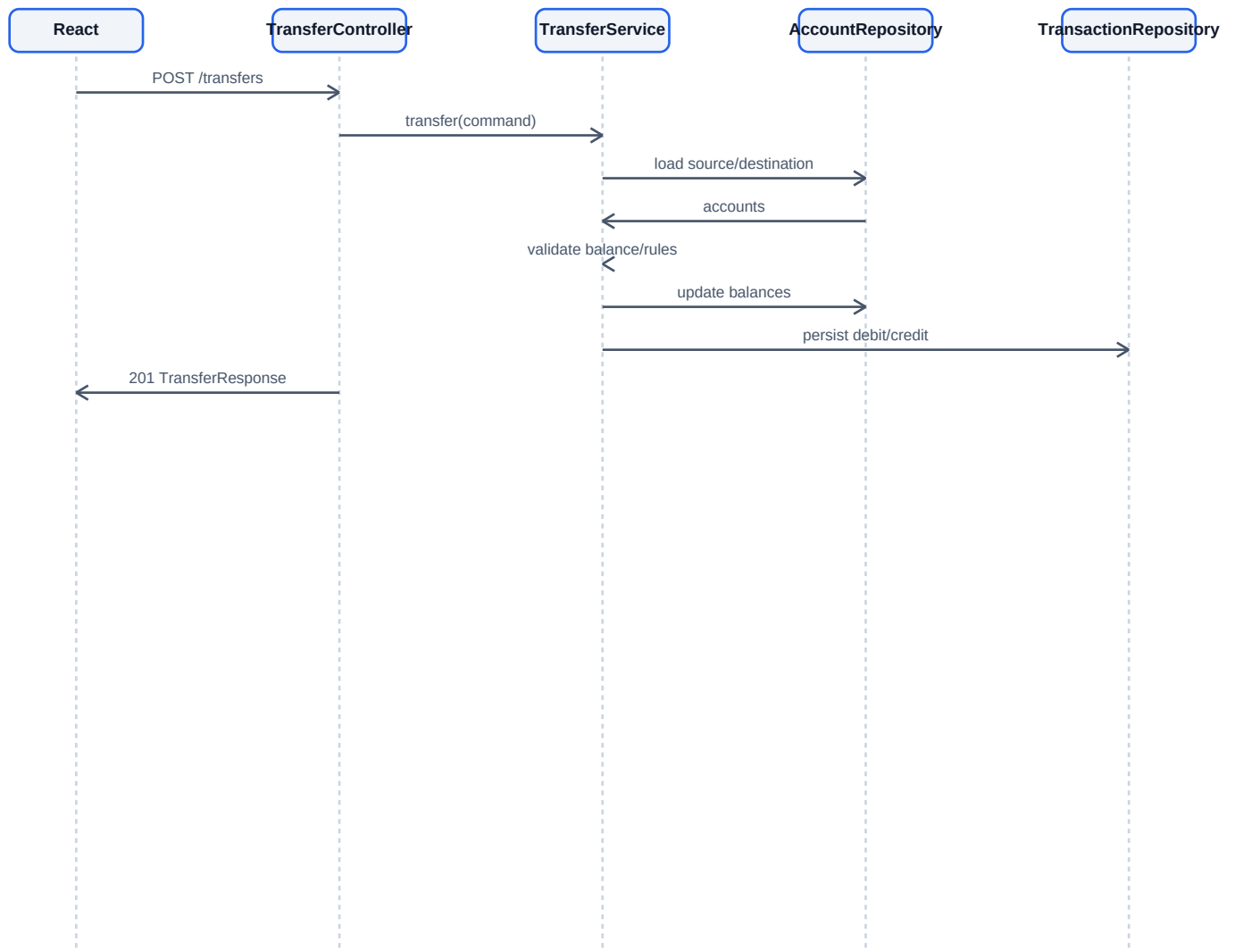
Entidades persistidas

users, accounts, transactions, categories, cards, card_purchases, invoices, financial_goals, ai_conversations e ai_messages. Recomenda-se UUID ou Long conforme a estratégia adotada; valores monetários devem usar BigDecimal e timestamps devem ser armazenados em UTC.

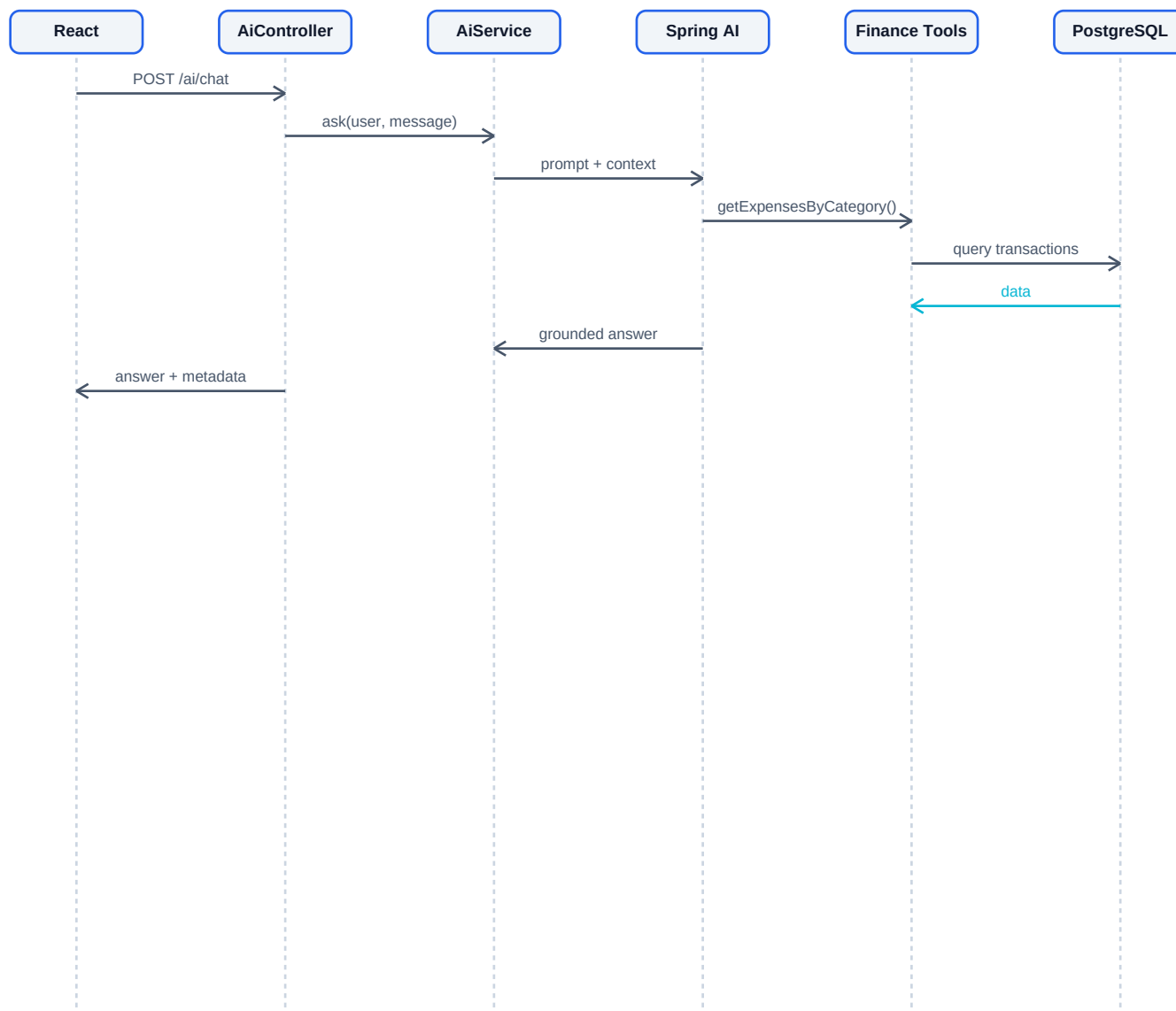
6. Fluxo de autenticação — sequência



7. Fluxo de transferência — sequência



8. Fluxo do Assistente FinAI — Spring AI + Tool Calling



Tools recomendadas

- `getCurrentBalance(accountId)`
- `getTransactions(accountId, period, filters)`
- `getExpensesByCategory(accountId, category, period)`
- `getFinancialGoals(userId)`
- `getCreditCardInvoice(cardId, month, year)`
- `getCashFlow(accountId, period)`

9. Contratos REST sugeridos

Módulo	Método	Endpoint	Objetivo
Auth	POST	/api/v1/auth/login	Autenticar e emitir JWT
Account	GET	/api/v1/accounts/{id}	Consultar conta/saldo
Transactions	GET	/api/v1/transactions	Listar transações
Transactions	POST	/api/v1/transactions	Criar lançamento
Transfers	POST	/api/v1/transfers	Realizar transferência
Cards	GET	/api/v1/cards	Listar cartões
Invoices	GET	/api/v1/cards/{id}/invoices	Consultar faturas
Goals	GET/POST	/api/v1/goals	Consultar/criar metas
Reports	GET	/api/v1/reports/cash-flow	Fluxo de caixa
AI	POST	/api/v1/ai/chat	Conversar com o FinAI

Padrões

- Versionar API em /api/v1.
- Usar DTOs de entrada/saída; não expor entidades JPA diretamente.
- Erros padronizados com Problem Details / RFC 9457 ou contrato equivalente.
- Paginação em listas e filtros por período/categoria.
- OpenAPI/Swagger como fonte navegável da API.

10. Dependências e responsabilidades

Tecnologia	Onde	Responsabilidade
React + TypeScript	Frontend	UI, estado de tela e interação
Vite	Frontend	Build e desenvolvimento
React Router	Frontend	Rotas protegidas e navegação
TanStack Query	Frontend	Cache, fetching e sincronização de dados
Tailwind CSS / UI kit	Frontend	Sistema visual e componentes
Java 21/25	Backend	Runtime e linguagem
Spring Boot	Backend	API e configuração
Spring Web	Backend	REST
Spring Security	Backend	Auth e autorização
Spring Data JPA	Backend	Persistência
PostgreSQL Driver	Backend	Conexão JDBC
Flyway	Backend	Migrações
Spring AI	Backend	Orquestração com LLM e tools
JUnit + Mockito	Backend	Testes
Testcontainers	Backend	Testes de integração com containers
Docker	Infra	Ambientes reprodutíveis

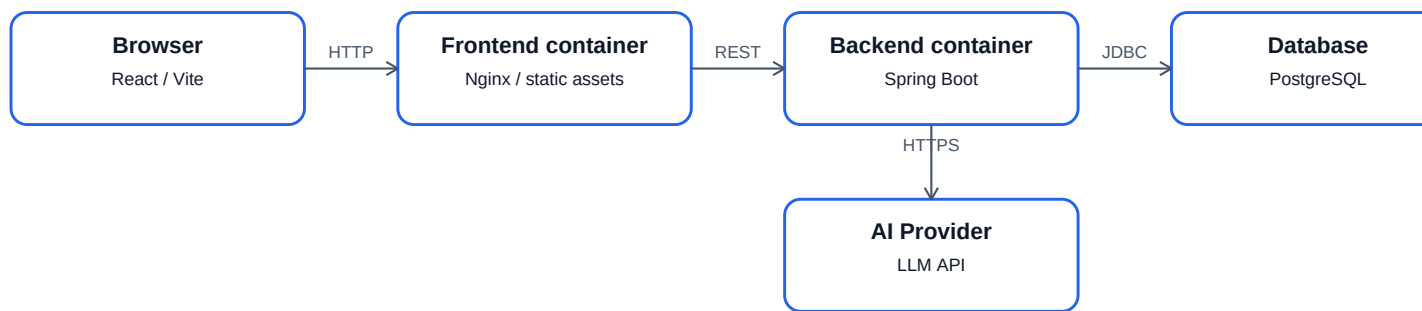
11. Segurança e regras arquiteturais

- Senhas nunca são armazenadas em texto puro; usar algoritmo de hashing apropriado.
- JWT com expiração curta e estratégia definida para refresh/revogação.
- Toda operação financeira deve verificar o ownership do recurso.
- Transferências e alterações de saldo devem ocorrer dentro de transação de banco.
- Valores monetários com BigDecimal; nunca usar double/float para dinheiro.
- Logs não devem registrar senha, token, número completo de cartão ou dados sensíveis.
- Ferramentas da IA recebem apenas identificadores autorizados pelo usuário autenticado.
- Prompts devem evitar instruções que permitam à LLM executar operações financeiras sem confirmação e regras de negócio.
- CORS, rate limiting e validação de entrada devem ser configurados antes de exposição pública.

Regra crítica do assistente

Perguntas informativas podem consultar dados. Operações que alterem estado financeiro — por exemplo, transferência, pagamento ou alteração de dados — devem passar por um fluxo explícito e autorizado da aplicação, não por uma decisão autônoma da LLM.

12. Arquitetura de desenvolvimento e deploy



Ambientes

- **Local:** React/Vite + Spring Boot + PostgreSQL em Docker.
- **CI:** build, lint, testes unitários e integração, análise estática.
- **Staging:** ambiente semelhante a produção com dados de teste.
- **Produção:** containers, banco gerenciado, HTTPS, secrets e observabilidade.

13. Estrutura de repositório

```
finai/  
  ■■■ backend/  
  ■ ■■■ src/main/java/com/finai/  
  ■ ■■■ domain/  
  ■ ■■■ application/  
  ■ ■■■ infrastructure/  
  ■ ■■■ api/  
  ■ ■■■ src/main/resources/  
  ■ ■■■ db/migration/  
  ■ ■■■ application.yml  
  ■ ■■■ pom.xml  
  ■■■ frontend/  
  ■ ■■■ src/  
  ■ ■■■ features/  
  ■ ■■■ components/  
  ■ ■■■ hooks/  
  ■ ■■■ services/  
  ■ ■■■ types/  
  ■ ■■■ app/  
  ■ ■■■ package.json  
  ■■■ docs/  
  ■ ■■■ uml/  
  ■ ■■■ arquitetura/  
  ■ ■■■ fluxos/  
  ■ ■■■ api/  
  ■■■ docker-compose.yml  
  ■■■ README.md  
  ■■■ .github/workflows/
```

Documentos recomendados

- README.md — visão geral e como executar.
- docs/arquitetura.md — decisões e componentes.
- docs/uml/casos-de-uso.puml — casos de uso.
- docs/uml/classes.puml — domínio.
- docs/uml/er.puml — banco.
- docs/fluxos/*.puml — sequências críticas.
- docs/api.md — contratos e convenções.
- docs/deciso-es-tecnicas.md — ADRs.

14. Roadmap de implementação

Fase	Entrega	Resultado
1	Setup	Repos, Maven, React/Vite, Docker e PostgreSQL
2	Identidade	User, login, JWT, roles e proteção
3	Conta	Account, saldo e transações
4	Financeiro	Categorias, relatórios, metas
5	Cartões	Cards, compras, parcelas e faturas
6	Frontend	Dashboard, extrato, gráficos e UX
7	IA	Spring AI, tools, chat e contexto
8	Qualidade	Testes, Testcontainers, validação e erros
9	Produção	Docker, CI/CD, observabilidade e segurança

Definition of Done inicial

- Feature possui endpoint, regra de negócio, persistência e testes.
- Frontend possui loading, erro, vazio e sucesso.
- Documentação/diagrama é atualizado junto com mudanças arquiteturais.
- Nenhuma operação financeira depende exclusivamente da resposta de uma LLM.